

Microcontrollers –BEC405AModule 1Module -1

RBT Level : L1, L2

Microcontroller: Microprocessor Vs Microcontroller, Microcontroller & Embedded Processors, Processor Architectures-Harvard Vs Princeton & RISC Vs CISC , 8051 Architecture- Registers, Pin diagram, I/O ports functions, Internal Memory organization. External Memory (ROM & RAM) interfacing. (Text book 1-1.1, Text book 2-1.0, 1.1, 3.0, 3.1, 3.2, 3.3 Text book 3-Pg 5-9)

Microcontroller:

A microcontroller is a highly integrated single chip, which consists of on chip CPU (Central Processing Unit), RAM (Random Access Memory), EPROM/PROM/ROM (Erasable Programmable Read Only Memory), I/O (input/output) – serial and parallel, timers, interrupt controller mainly designed for control applications.

Applications of microcontroller**1. Consumer Electronics:**

Washing machine, Microwave oven, Dish washer, toys, remote controls, coffee machines, refrigerators, alarm clocks, air conditioners, Digital cameras, Mobile phones etc.

2. Office:

Fax machines, printers, copiers, security systems, paging and intercom, ATM machines, CCTV camera and surveillance and elevators etc.

3. Automobiles:

Engine fuel control, Automatic braking system(ABS), air bag control, centre locking system, power windows and seats etc.

4. Electronic measurement instruments:

Digital multimeters, frequency synthesizers and oscilloscopes, logic analyzers, spectrum analyzers, digital thermometers, tachometers etc.

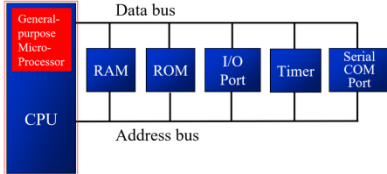
5. Biomedical systems:

ECG recorder, blood cell analyzers, glucose monitor, patient monitoring systems etc.

6. Others:

Musical instruments, Electronic voting machines, traffic controller, vehicle tracking etc.

Microprocessor Vs Microcontroller

Microprocessor	Microcontroller
A silicon chip representing a Central Processing Unit (CPU), which is capable of performing arithmetic as well as logical operations according to a pre-defined set of Instructions	A microcontroller is a highly integrated chip that contains a CPU, scratch pad RAM, Special and General purpose Register Arrays, On Chip ROM/FLASH memory for program storage, Timer and Interrupt control units and dedicated I/O ports
It is a dependent unit. It requires the combination of other chips like Timers, Program and data memory chips, Interrupt controllers etc for functioning	It is a self contained unit and it doesn't require external Interrupt Controller, Timer, UART etc for its functioning
Most of the time general purpose in design and operation	Mostly application oriented or domain specific
Doesn't contain a built in I/O port. The I/O Port functionality needs to be implemented with the help of external Programmable Peripheral Interface Chips like 8255	Most of the processors contain multiple built-in I/O ports which can be operated as a single 8 or 16 or 32 bit Port or as individual port pins
Targeted for high end market where performance is important	Targeted for embedded market where performance is not so critical (At present this demarcation is invalid)
Limited power saving options compared to microcontrollers	Includes lot of power saving features
	

Criteria for choosing a microcontroller

- ❖ Speed : Highest speed supported by the microcontroller
- ❖ Power Consumption: Low power microcontrollers are preferred for battery operated embedded systems
- ❖ Packaging : Dual in Package (DIP), Quad flat package (QFP)
- ❖ Memory Requirement : Amount of on chip RAM, ROM
- ❖ The number of IO pins & on chip peripherals
- ❖ Cost of the Microcontroller

Microcontroller vs Embedded Processors

Microcontrollers	Embedded Processors
Microcontroller is a highly integrated chip that includes a CPU (Central Processing Unit), memory (both volatile RAM and non-volatile ROM or Flash), and various peripherals (such as timers, serial communication interfaces, analog-to-digital converters, etc.) all on a single chip	An embedded microprocessor, on the other hand, is a standalone processing unit that requires external components such as memory (RAM, ROM), I/O devices, and other peripherals to function effectively
Microcontrollers are designed to execute specific tasks within embedded systems without needing external components	Embedded processors are designed to execute specific tasks within embedded systems with the need of external peripheral components
Microcontrollers are typically used in applications where cost, power consumption, and size are critical factors.	Embedded microprocessors are often used in more complex systems where customization and flexibility are prioritized over cost and size considerations
Microcontrollers are commonly used in Consumer Electronics (TV, Camera, Washing machine, Microwave oven), Automotive systems (ECU, ABS, airbag control), robotics etc.	Embedded microprocessors are commonly found in applications like smartphones, tablets, automotive electronics, and industrial control systems.

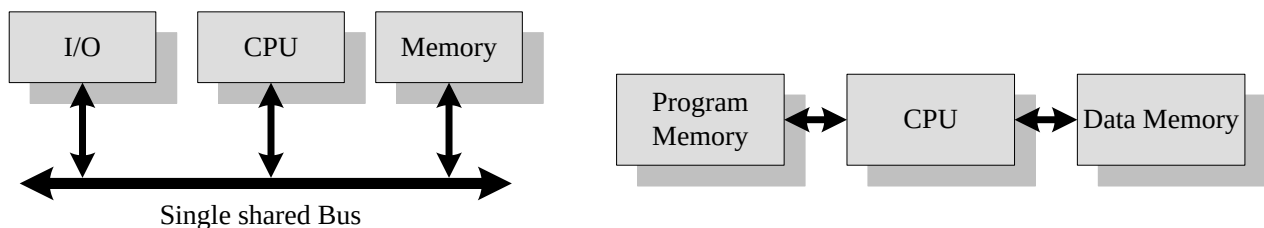
RISC vs CISC

RISC	CISC
Lesser no. of instructions	Greater no. of Instructions
Instruction Pipelining and increased execution speed	Generally no instruction pipelining feature
Orthogonal Instruction Set (Allows each instruction to operate on any register and use any addressing mode)	Non Orthogonal Instruction Set (All instructions are not allowed to operate on any register and use any addressing mode. It is instruction specific)
Operations are performed on registers only, the only memory operations are load and store	Operations are performed on registers or memory depending on the instruction
Large number of registers are available	Limited no. of general purpose registers
Programmer needs to write more code to execute a task since the instructions are simpler ones	Instructions are like macros in C language. A programmer can achieve the desired functionality with a single instruction which in turn provides the effect of using more simpler single instructions in RISC
Single, Fixed length Instructions	Variable length Instructions
Less Silicon usage and pin count	More silicon usage since more additional decoder logic is required to implement the complex instruction decoding.
Harvard Architecture	Can be Harvard or Von-Neumann Architecture

Harvard V/s Von-Neumann Processor/Controller Architecture

- ❖ The terms Harvard and Von-Neumann refers to the processor architecture design.
- ❖ Microprocessors/controllers based on the Von-Neumann architecture shares a single common bus for fetching both instructions and data. Program instructions and data are stored in a common main memory
- ❖ Microprocessors/controllers based on the Harvard architecture will have separate data bus and instruction bus. This allows the data transfer and program fetching to occur simultaneously on both buses

With Harvard architecture, the data memory can be read and written while the program memory is being accessed. These separated data memory and code memory buses allow one instruction to execute while the next instruction is fetched (“Pre-fetching”)



PIN DESCRIPTION OF 8051

Pins 1-8: Port 1 Each of these pins can be configured as an input or an output.

Pin 9: RST A logic one on this pin disables the microcontroller and clears the contents of most registers. In other words, the positive voltage on this pin resets the microcontroller. By applying logic zero to this pin, the program starts execution from the address 0000h. Upon reset all the registers except stack pointer will be cleared to 00h and stack pointer will be loaded with the address 07h.

Pins 10-17: Port 3 Similar to port 1, each of these pins can serve as general input or output. All the pins of port 3 have different alternative functions.

Pin 10: RXD Serial asynchronous communication input.(Receive Data)

Pin 11: TXD Serial asynchronous communication output.(Transmit Data)

Pin 12: INT0 Interrupt 0 input.

Pin 13: INT1 Interrupt 1 input.

Pin 14: T0 Counter 0 clock input.

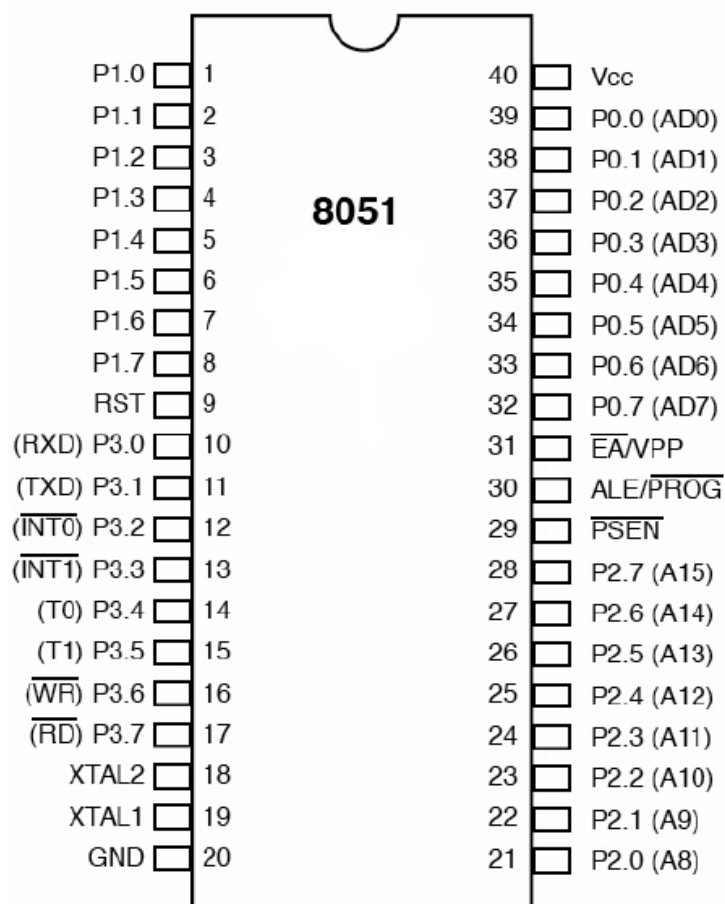
Pin 15: T1 Counter 1 clock input.

Pin 16: WR Write to external (additional) RAM.

Pin 17: RD Read from external RAM.

Pin 18, 19: XTAL2, XTAL1 Internal oscillator input and output. A quartz crystal with specific operating frequency is usually connected to these pins. Instead of it, miniature ceramics resonators can also be used for frequency stability.

Pin 20: GND Ground.



Pin 21-28: Port 2 If there is no intention to use external memory then these port pins are configured as general inputs/outputs. In case external memory is used, the higher address byte, i.e. addresses A8-A15 will appear on this port. Even though memory with capacity of 64Kb is not used, which means that not all eight port bits are used for its addressing, the rest of them are not available as inputs/outputs.

Pin 29: PSEN If external ROM is used for storing program then a logic zero (0) appears on it every time the microcontroller reads a byte from external ROM.

Pin 30: ALE Prior to reading from external memory, the microcontroller puts the lower address byte (A0-A7) on P0 and activates the ALE output. After receiving signal from the ALE pin, the external register (usually 74HCT373 or 74HCT375 add-on chip) memorizes the state of P0 and uses it as a memory chip address. Immediately after that, the ALU pin is returned its previous logic state and P0 is now used as a Data Bus. As seen, port data multiplexing is performed by means of only one additional (and cheap) integrated circuit. In other words, this port is used for both data and address transmission.

Pin 31: EA By applying logic zero to this pin, P2 and P0 are used for data and address transmission with no regard to whether there is internal memory or not. It means that even there is a program written in the onchip

memory of the microcontroller, it will not be executed. Instead, the program written in external ROM will be executed. By applying logic one to the EA pin, the microcontroller will use both memories, first internal ROM and then external ROM (if exists).

Pin 32-39: Port 0 Similar to P2, if external memory is not used, these pins can be used as general inputs/outputs. Otherwise, P0 is configured as address output (A0-A7) when the ALE pin is driven high (1) or as data output D0 – D7 (Data Bus) when the ALE pin is driven low (0).

Pin 40: VCC +5V power supply

Salient features of 8051:

- 8 bit CPU with registers A and B
- 16 bit Program Counter(PC) and Data pointer(DPTR).
- 8 bit program status word(PSW)
- 8 bit Stack Pointer(SP)
- 4K Internal ROM
- 128bytes Internal RAM
 - 4 register banks each having 8 registers
 - 16 bytes, which may be addressed at the bit level.
 - 80 bytes of general purpose data memory
- 32 input/output pins arranged as four 8 bit ports:P0 to P3
- Two 16 bit timer/counters: Timer0 and Timer1
- 8 bit full duplex serial data receiver/transmitter: UART
- Control registers: TCON, TMOD, SCON, PCON, IP and IE
- Two external and three internal interrupt sources.
- Oscillator and Clock Circuits

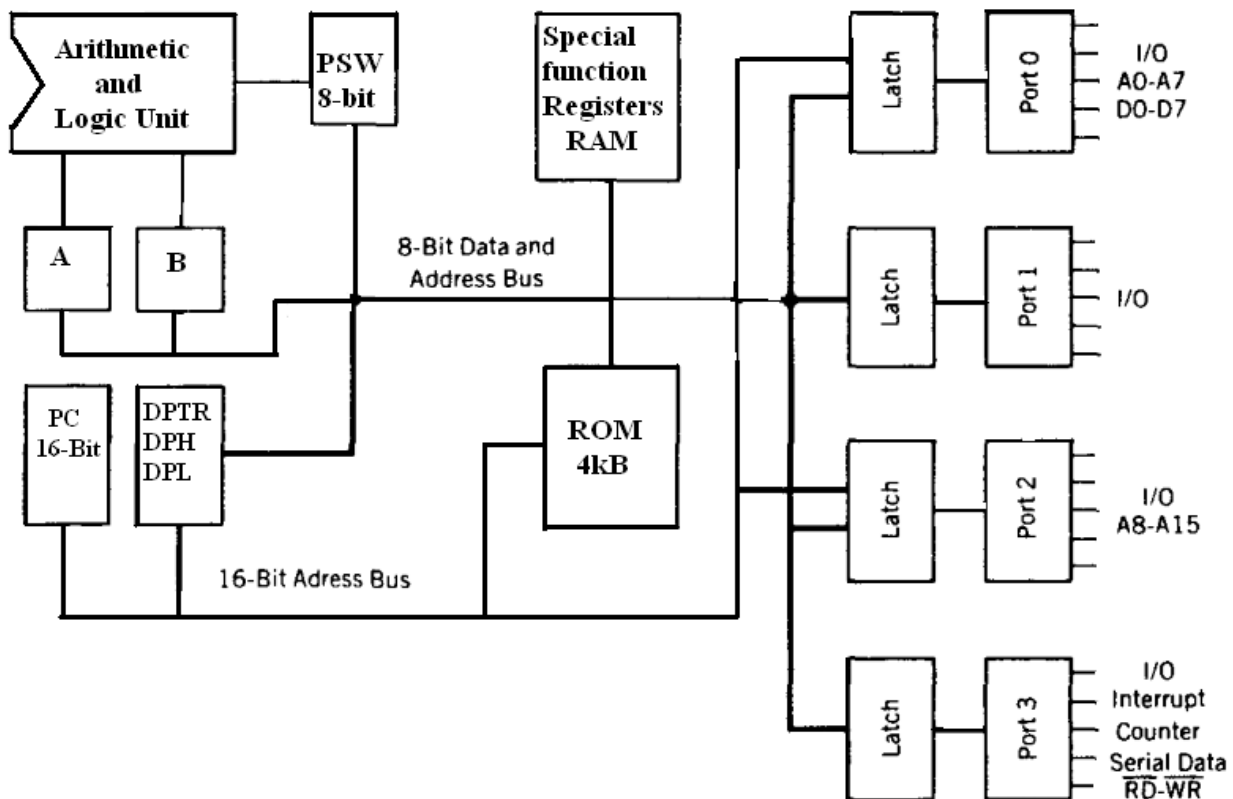
ARCHITECTURE OF 8051

The internal components of 8051 are defined as its architecture. The block diagram of 8051 microcontroller is shown in Figure above. It consists of an 8-bit ALU, one 8-bit PSW(Program Status Register), A and B registers, one 16-bit Program counter, one 16-bit Data pointer register(DPTR), 128 bytes of RAM and 4kB of ROM and four parallel I/O ports each of 8-bit width.

A, B and CPU registers:

There are totally 34 general purpose registers or working registers available in 8051. Two of these are A and B registers used to hold results of many instructions, particularly arithmetic and logical operations of 8051 CPU. The other 32 are in four banks, Register bank0 – Register bank3 of eight registers(R0 – R7) each. A (accumulator) is used as source and destination for addition, subtraction, multiplication, division, and for

data transfers. But B register is used to store the higher byte of the multiplied result in multiplication and to store remainder in division operations.

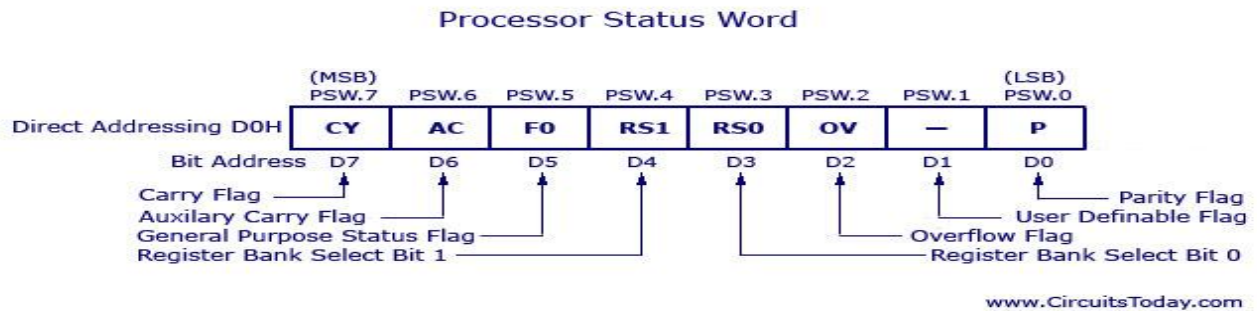


Program counter(PC) and Data pointer(DPTR):

Both are 16 bit registers to hold the address of a byte in memory. PC contains the address of the next instruction to be executed. The contents of program counter is automatically incremented after execution of instruction. DPTR is made up of two 8 bit registers DPH and DPL. DPTR is used to hold the address of internal & external code and data memory that has to be accessed.

Program status word(PSW)

The program status word (PSW) register is an 8-bit register. It is also referred to as the *flag register*. Although the PSW register is 8 bits wide, only 6 bits of it are used by the 8051. The two unused bits are user-definable flags. Four of the flags are called *conditional flags*, meaning that they indicate some conditions that result after an instruction is executed. These four are CY (carry), AC (auxiliary carry), P (parity), and OV (overflow). As seen from below figure, the bits PSW.3 and PSW.4 are designated as RS0 and RS1 as register selection bit, respectively, and are used to change the bank registers. The PSW.5 and PSW.1 bits are general-purpose status flag bits and can be used by the programmer for any purpose. In other words, they are user definable. Below Figure shows the individual bits of the PSW register.

**Carry flag (C):**

Used to check whether the result is within the range of 8 bit unsigned numbers after unsigned arithmetic operations.

- Set to logic 1 if a carry generated from bit D7 of the result
- Reset to logic 0 if a carry is not generated from bit D7 of the result

Auxiliary carry flag(AC):

Used to check the correctness of the result in BCD arithmetic operation

- Set to logic 1 if a carry generated from bit D3 to bit D4 of the result
- Reset to logic 0 if a carry is not generated from bit D3 to bit D4 of the result

Overflow flag(OV)

Used to check whether the result is within the range of signed numbers in signed arithmetic operation

- set to logic 1 if a carry generated from bit D6 to bit D7 and no carry from D7 (or) if carry generated from bit D7 and no carry from bit D6 to bit D7, else it is reset to logic 0.

Parity flag - (P)

Used for error detection and correction in serial data communication

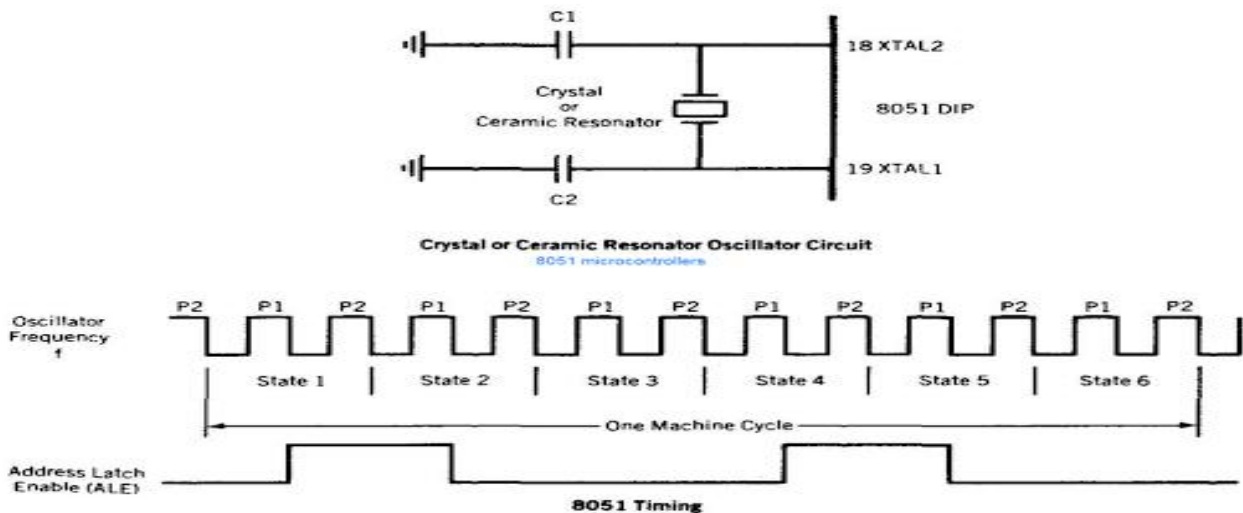
- set to logic 1 if the result has odd number of 1's and reset to logic 0 if the result has even number of 1's.

Register bank select bits(RS0, RS1)

RS1	RS0	Register Bank	Register Bank Status
0	0	0	Register Bank 0 is selected
0	1	1	Register Bank 1 is selected
1	0	2	Register Bank 2 is selected
1	1	3	Register Bank 3 is selected

8051 oscillator and clock:

The heart of 8051 microcontroller is the circuit that generates the clock pulses by which all the internal operations are synchronized. Pins XTAL1 & XTAL2 are used for connecting resonant circuit to form an oscillator. Generally a quartz crystal is connected along with capacitors because of its high stability factor. The crystal frequency is the basic internal clock frequency of 8051. General operating crystal frequency of 8051 is 1MHZ to 16MHZ. Generally crystal frequency of 11.0592 MHZ is used to support serial communication baud rates.



The clock frequency establishes the smallest interval of time within the microcontroller called period or pulse. The smallest interval of time that is required to execute a simple instruction or a part of complex instruction is called **machine cycle**. Each machine cycle of 8051 itself is made up of 6 states. A state is discrete time interval required for microcontroller to do some discrete operations like fetching opcode, decoding an opcode, fetching of data byte and execution of instruction etc. Two oscillator periods form a state.

All instructions of 8051 generally require 1 or 2 or 4 machine cycles to execute depending on the type of instruction. Instructions are fetched and executed automatically starting from the address 0000h of the program memory. The time taken by microcontroller to execute an instruction is called instruction cycle. Time taken to execute an instruction is calculated by multiplying C with 12 and then divide the product by crystal frequency (i.e)

$T_{inst} = (C \times 12) / \text{crystal frequency}$ ---- where C is the number of machine cycle used by an instruction.

For eg. If the crystal frequency used is 12MHz, then time taken to execute an instruction ADD A, R1 with 1 machine cycle is $(1 \times 12) / 12\text{MHz}$ is 1μs.

Timers/ Counters:

8051 core has two 16 bit timers timer0 and timer1. These timers are used for generation of delays in between two different events and to count some external events. Each 16 bit timer can also be used as 2 8 bit timers TH0, TL0 and TH1, TL1. These two timers will be controlled using TMOD and TCON.

TMOD : Timer/Counter Mode Control Register (Not Bit Addressable)

GATE	C/T	M1	M0	GATE	C/T	M1	M0
TIMER 1				TIMER 0			

TCON : Timer/Counter Control Register (Bit Addressable)

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

Serial communication UART:

8051 core supports inbuilt 8 bit universal asynchronous receiver transmitter for serial communication. SBUF is an eight bit register used to hold the 8 bit data before transmitting the data serially, and after receiving the data serially. SCON is an 8 bit register used to control serial communication.

SCON register (Bit addressable):

SM0	SM1	SM2	REN	TB8	RB8	TI	RI
-----	-----	-----	-----	-----	-----	----	----

Interrupt control:

An interrupt is an external or internal event that disturbs the microcontroller to inform it that a device needs its service. The program which is associated with the interrupt is called the interrupt service routine (ISR) or interrupt handler. Upon receiving the interrupt signal the Microcontroller, finish current instruction and saves the PC on stack. Jumps to a fixed location in memory depending on type of interrupt. Starts to execute the interrupt service routine until RETI (return from interrupt). Upon executing the RETI the microcontroller returns to the place where it was interrupted. The 8051 microcontroller has FIVE interrupts in addition to Reset. They are

1. Timer 0 overflow Interrupt
2. Timer 1 overflow Interrupt
3. External Interrupt 0 (INT0)
4. External Interrupt 1 (INT1)
5. Serial Interrupt

Each interrupt has a specific place in code memory where program execution (interrupt service routine) begins.

External Interrupt 0: 0003 H

Timer 0 overflow: 000B H

External Interrupt 1: 0013 H

Timer 1 overflow: 001B H

Serial Interrupt : 0023 H

Upon reset all Interrupts are disabled & do not respond to the Microcontroller. These interrupts must be enabled by software in order for the Microcontroller to respond to them. This is done by an 8-bit register called Interrupt Enable Register (IE).

Parallel I/O ports:

8051 core supports 4 parallel I/O ports port0(P0), port1(P1), port2(P2) and port3(P3) of size 8 bits. Upon reset these ports are default configured as output ports. These ports are configured as input by writing logic 1 to each port pin. For example port0 can be configured as input using the instruction MOV P0, #0FFh.

Port 0:

Port 0 is an 8-bit I/O port with dual purpose. If external memory is used, these port pins are used for the lower address byte address/data (AD0-AD7), otherwise all bits of the port are either input or output.. Unlike other ports, Port 0 is not provided with pull-up resistors internally ,so for PORT0 pull-up resistors of nearly 10k are to be connected externally.

Port 1:

Port 1 occupies a total of 8 pins (pins 1 through 8). It has no dual application and acts only as input or output port. In contrast to port 0, this port does not need any pull-up resistors since pull-up resistors connected internally.

Port 2 :

Port 2 is also an eight bit parallel port. (pins 21- 28). It can be used as input or output port and also as higher order address lines (A8 – A15). As this port is provided with internal pull-up resistors it does not need any external pull-up resistors.

Port 3 :

Port3 is also an 8-bit parallel port with distinct dual function for each pin . The port pins can be used for I/O operations as well as for control operations

Internal memory Organisation:

8051 supports internal RAM of size 128 bytes with address space 00h-7Fh and internal ROM of size 4KB with address space 000h – FFFh.

Internal RAM:

Internal RAM of size 128 bytes is found on-chip on the 8051. Since RAM is volatile, this memory is cleared after the power is off. The 128 bytes of internal RAM is organized as below.

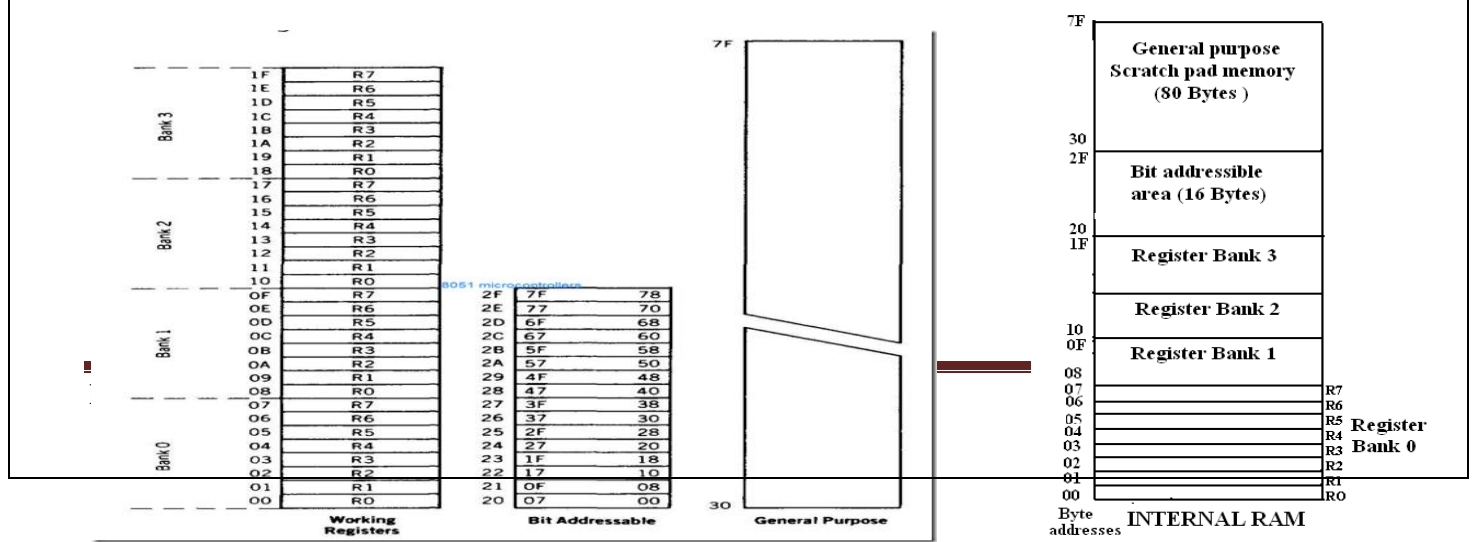
(i) The first 32 bytes of RAM from address 00 H to 1FH are used as working registers organized as four register banks of eight registers each. The registers are named as R0-R7 .Each register can be addressed by its name or by its RAM address.

For Eg. : MOV R7,#05H

MOV A, R7

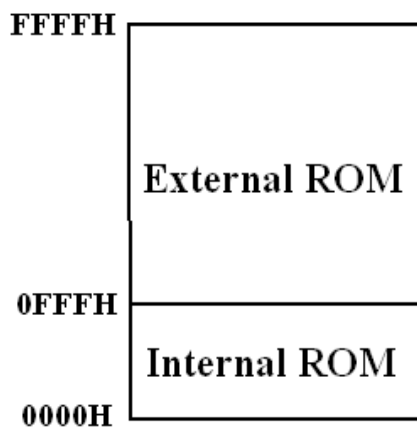
The default register bank under select is Bank0. The remaining Banks are selected with the help of RS0 and RS1 bits of PSW Register.

(ii) Next 16 bytes are bit addressable area with address range 20h-2Fhand the rest 80 bytes are used as general purpose area with address range 30h – 7Fh(Scratch pad memory) as shown in the diagram below.

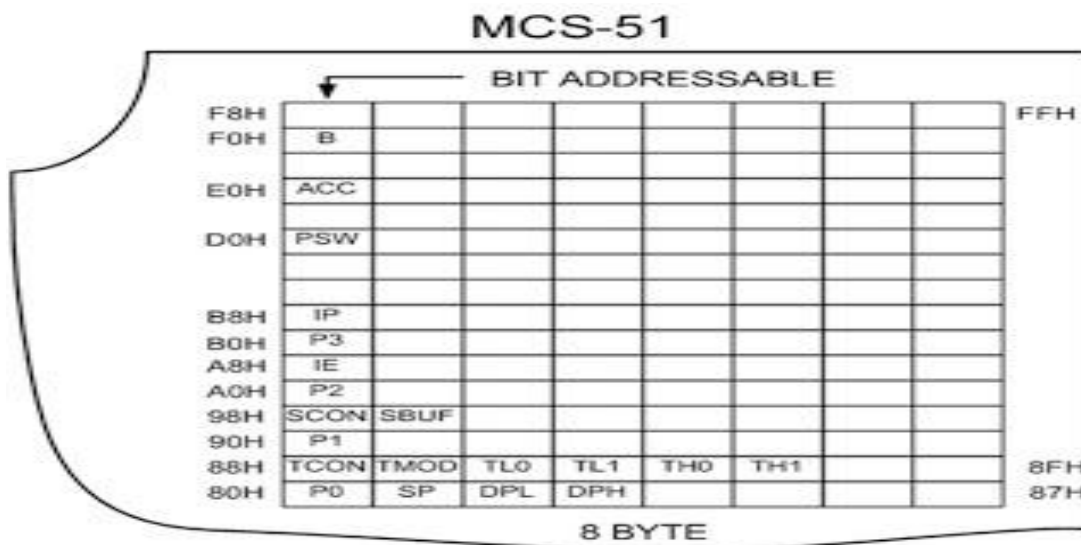


Internal ROM (On –chip ROM):

The 8051 microcontroller has 4kB of on chip ROM with address space of 0000h to 0FFFh, but it can be extended up to 64kB with external ROM. This ROM is also called program memory or code memory. The program instructions from this memory are accessed using the program counter (PC) and on chip data are fetched using DPTR. The external ROM is accessed when the EA(active low) pin is connected to ground or the contents of program counter exceeds 0FFFh. When the content of program counter exceeds 0FFFh the 8051 automatically fetches the code bytes from the external program memory if EA pin is connected to VCC.

**SPECIAL FUNCTION REGISTERS (SFRs) :**

In 8051 microcontroller there are certain registers which use the address space from 80h to FFh and they are meant for certain specific operations. These registers are called Special function registers (SFRs). Some of these registers are bit addressable also. The list of SFRs and their functional names are given below. In these SFRs some of them are related to I/O ports (P0, P1, P2 and P3) and some of them are meant for control operations (TCON, SCON, PCON, IE, IP, TMOD)

**STACK****MEMORY ORGANIZATION:**

The stack is apart of RAM used by the CPU to store information temporarily. This information may be either data or an address. The CPU needs this storage area as there are only limited number of registers. The register used to access the stack is called the Stack pointer which is an 8-bit register. When the 8051 is powered up, the SP register contains the value 07h. Generally stack uses the address space 08h to 1Fh and it may extend to the memory area from 30h to FFh if excess stack area is required.

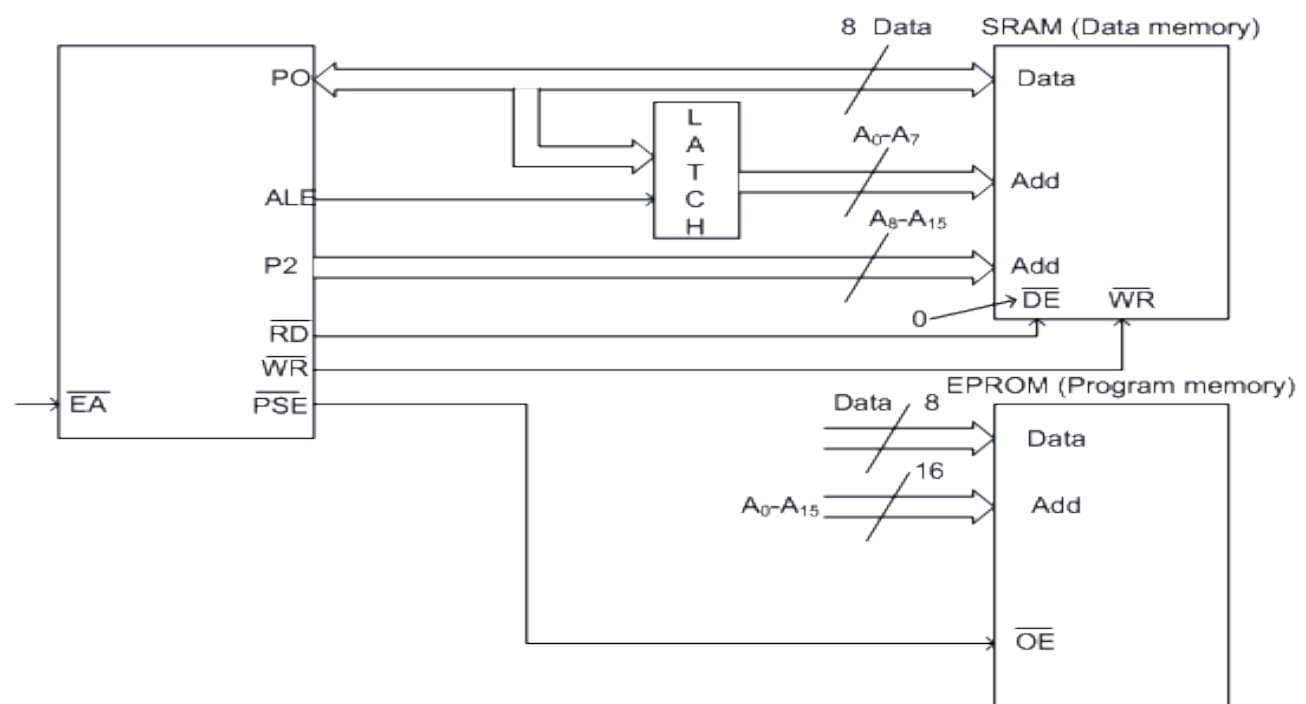
There are two important instructions to handle this stack. One is **PUSH source** and the Other is **POP destination**. The loading of data from CPU registers & general purpose memory area to the stack is done by PUSH and the loading of the contents of the stack back into a CPU registers & general purpose memory area is done by POP.

EX : MOV R6, #35h

PUSH 6

In the above instructions the contents of the Registers R6 is moved to stack memory address 08h by incrementing the stack pointer(SP) by 1 automatically from its reset value 07h. Similarly POP 5 instruction pops the contents of stack memory pointed by stack pointer into R5 register and then the contents of the SP is decremented by 1.

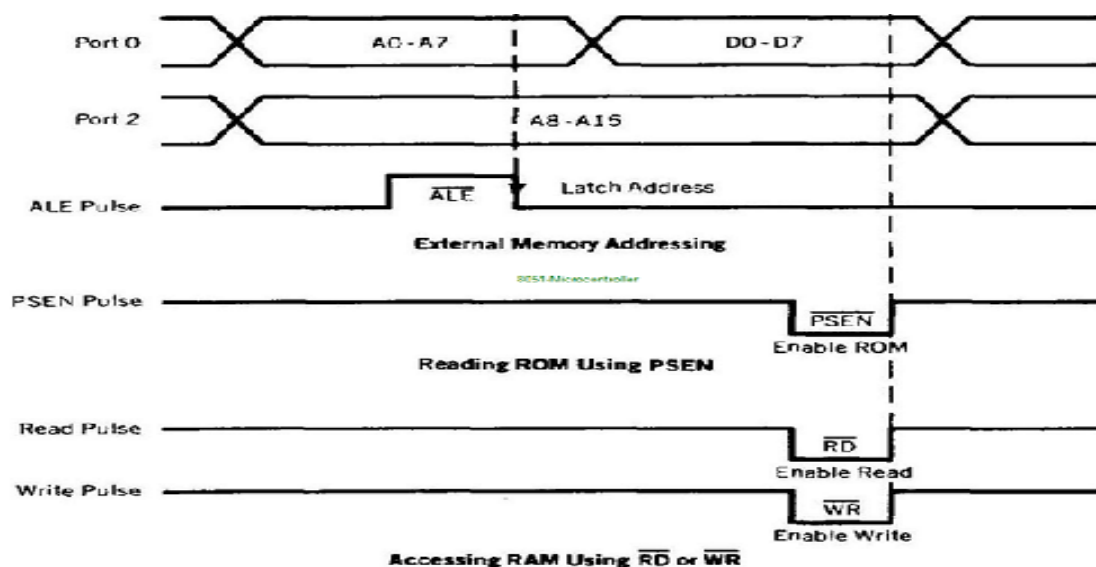
External memory Interfacing:



64KB of external RAM & 64KB external ROM can be connected with 8051, since it supports Harvard architecture. Port P0 of 8051 acts as lower order address lines or data lines and Port P2 acts as higher order address lines when external memory is interfaced with 8051. Port P0 is connected with the data lines of

external RAM & ROM directly and via a latch 74LS373 to the address lines of external RAM & ROM. ALE signal acts as an enable signal of the latch. Whenever microcontroller asserts ALE signal to logic high, latch gets enabled and it latches the address information to the address lines of memory. When 8051 asserts the ALE signal with logic low, the latch gets disabled and Port P0 acts as databus to the external memory. RD & WR signals of 8051 are connected to RD & WR signals of external RAM. 8051 reads the data from the external RAM through the data lines by asserting the RD signal with logic 0 and writes the data to external RAM by asserting the WR signal to logic 0.

Program store Enable (PSEN) signal of 8051 is connected to the output enable (OE) of external ROM. The code or data from the external ROM is read by the microcontroller by asserting the PSEN signal to logic 0. Read and write timing diagram of both external RAM & ROM is shown below.



Additional Notes

Embedded Systems

Embedded system is a combination of processing hardware and application software that is designed for a specific function or for a specific function within a larger system. One of the most critical needs of an embedded system is to decrease power consumption and space.

Embedded Microcontrollers

An embedded system uses a microcontroller (or microprocessor) to do the specific designed task. Generally microcontrollers of 8 bit, 16-bit and 32-bit made by various chip makers are used as a processing device in embedded systems. Many manufactures of general-purpose microprocessors have targeted their microprocessor for the high end of the embedded market during the times that a microcontroller is inadequate

for the task. Very often the terms embedded processor and microcontroller are used interchangeably. Some of the 8-bit microcontrollers used in embedded products are

- Motorola's 6811
- Intel's 8051
- Zilog's Z8
- Microchip's PIC 16C

The selection of microcontroller to meet the computing needs of the task at hand efficiently and cost effectively is based on

- Speed
- Packaging
- Power consumption
- The amount of RAM and ROM on chip
- The number of I/O pins and the timer on chip
- How easy to upgrade to higherperformance or lower power-consumption versions
- Cost per unit
- Availability of software development tools, such as compilers, assemblers, and debuggers.

MICROCONTROLLER APPLICATION DEVELOPMENT TOOLS:

To develop an assembly language program we need certain program development tools. An assembly language program consists of Mnemonics which are nothing but short abbreviated English instructions given to the controller. The various development tools required for Microcontroller programming are explained below.

1. Editor : An Editor is a program which allows us to create a file containing the assembly language statements for the program. Examples of some editors are PC write Wordstar. After typing all the instructions of a program we have to save the program. This we call it as source file. The next step is to process the source file with an assembler.

Ex: Sample.asm

2.Assembler : An Assembler is used to translate the assembly language mnemonics into machine language (i.e binary codes). When you run the assembler it reads the source file of your program from where you have saved it. The assembler generates a file with the extension **.hex**. This file consists of hexadecimal values encoding a sequence of data and their starting offset or absolute address.

3.Compiler : A compiler is a program which converts the high level language program like "C" into binary or machine code. Using high level languages is easy to manage complex data structures which are often required for data manipulation. Because of its ease, flexibility and debug options now a days the compilers have become very popular in the market. Compilers like Keil, Ride and IAR workbench are very popular.

3. Debugger/Simulator : A debugger is a program which allows to execute the program, and troubleshoot or debug it. The debugger allows to look into the contents of registers and memory locations after the program runs. We can also change the contents of registers and memory locations and rerun the program. Some debuggers allows to stop the program after each instruction so that you can check or alter memory and register contents. This is called single step debug. A debugger also allows to set a breakpoint at any point in the program. If we insert a break point , the debugger will run the program up to the instruction where the breakpoint is put and then stop the execution.

A simulator is a software program which virtually executes the instructions similar to a microcontroller and shows the results. This will help in evaluating the results without committing any errors. By doing so we can detect the possible logic errors